A1 - DAA

```
// Fibonacci.java

import java.util.Scanner;
public class Fibonacci {
    // recursive
    static int fibonacciRecursive(int n){
        if (n <= 1){
            return n;
        }
        return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
    }
    //nonrecursive
    static void fibonacciIterative(int n) {
        int a = 0;
        int b = 1;
        for (int i = 0; i < n; i++){
            System.out.print(a + " ");
            int next = a + b;
            a = b;
            b = next;
        }
        System.out.println();
    }
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter NO OF TERMS: ");
        int n = sc.nextInt();
        if ( n < 0){
            System.out.println(" NO OF TERMS CANNOT BE NEGATIVE");
            sc.close();
            return;
        }
        System.out.println("\nFIBONACCI USING RECURSION: ");
        for (int i = 0; i < n; i++){
            System.out.print(fibonacciRecursive(i) + " ");

                }
                System.out.println("\n\nFibonacci using iteration:");

        fibonacciIterative(n);

        sc.close();
    }
}
```

A2 - DAA

```
// HuffmanCoding.java

import java.util.PriorityQueue;
import java.util.Scanner;

class Node {
    char ch;
    int frequency;
    Node left, right;

    Node(char ch, int frequency) {
        this.ch = ch;
        this.frequency = frequency;
        left = right = null;
    }
}

public class HuffmanCoding {

    static void printCodes(Node root, String code) {
        if (root == null)
            return;

        // Leaf node represents a character
        if (root.left == null && root.right == null) {
            System.out.println(root.ch + " : " + code);
            return;
        }

        printCodes(root.left, code + "0");
        printCodes(root.right, code + "1");
    }

    static void buildHuffmanTree(char[] characters, int[] frequencies) {
        PriorityQueue<Node> pq = new PriorityQueue<>(
            (a, b) -> Integer.compare(a.frequency, b.frequency)
        );

        // Insert all characters into the priority queue
        for (int i = 0; i < characters.length; i++) {
            pq.add(new Node(characters[i], frequencies[i]));
        }

        // Build the Huffman tree
        while (pq.size() > 1) {
            Node left = pq.poll();
            Node right = pq.poll();

            Node parent = new Node(
                '\0',
                left.frequency + right.frequency
            );

            parent.left = left;
            parent.right = right;

            pq.add(parent);
        }

        Node root = pq.poll();

        System.out.println("\nHuffman Codes:");

        if (root.left == null && root.right == null) {
            System.out.println(root.ch + " : 0");
        } else {
            printCodes(root, "");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of characters: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Number of characters must be positive.");
            sc.close();
            return;
```

```
        }

        char[] characters = new char[n];
        int[] frequencies = new int[n];

        System.out.println("Enter each character followed by its frequency:");

        for (int i = 0; i < n; i++) {
            characters[i] = sc.next().charAt(0);
            frequencies[i] = sc.nextInt();

            if (frequencies[i] <= 0) {
                System.out.println("Frequency must be positive.");
                sc.close();
                return;
            }
        }

        buildHuffmanTree(characters, frequencies);
        sc.close();
    }
}
```

A3 - DAA

```
// KnapsackDP.java

import java.util.Scanner;
public class KnapsackDP {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("ENTER NO OF ITEMS:");
        int n = sc.nextInt();

        int[] weights = new int[n];
        int[] profit = new int[n];
        System.out.println("ENTER WEIGHTS OF ITEMS:");
        for (int i = 0; i < n; i++){
            weights[i] = sc.nextInt();
        }
        System.out.println("ENTER PROFITS OF ITEMS:");
        for (int i = 0; i < n; i++){
            profit[i] = sc.nextInt();
        }
        System.out.print("ENTER CAPACITY OF KNAPSACK:");
        int capacity = sc.nextInt();

        int[][] dp = new int[n + 1][capacity + 1];
        for (int i = 1; i <=n; i++){
            for (int w = 1; w <= capacity; w++){
                if (weights[i - 1] <= w){
                    dp[i][w] = Math.max(dp[i - 1][w], profit[i - 1] + dp[i - 1][w - weights[i -
 1]]);
                } else {
                    dp[i][w] = dp[i - 1][w];
                }
            }
        }
        System.out.println("MAXIMUM PROFIT:" + dp[n][capacity]);
                System.out.print("Selected Items: ");

        int w = capacity;

        for (int i = n; i > 0; i--) {

            if (dp[i][w] != dp[i - 1][w]) {

                System.out.print(i + " ");

                w = w - weights[i - 1];

            }

        }

        System.out.println();

        sc.close();

    }

}
```

A4 - DAA

```
// NQueens.java
import java.util.Scanner;

public class NQueens {

    // Display the chessboard
    static void printBoard(int[][] board, int n) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == 1) {
                    System.out.print("Q ");
                } else {
                    System.out.print(". ");
                }
            }
            System.out.println();
        }

    }

    // Check whether placing a queen is safe
    static boolean isSafe(int[][] board, int row, int col, int n) {

        // Check same column in previous rows
        for (int i = 0; i < row; i++) {
            if (board[i][col] == 1) {
                return false;
            }
        }

        // Check upper-left diagonal
        int i = row - 1;
        int j = col - 1;

        while (i >= 0 && j >= 0) {
            if (board[i][j] == 1) {
                return false;
            }
            i--;
            j--;
        }

        // Check upper-right diagonal
        i = row - 1;
        j = col + 1;

        while (i >= 0 && j < n) {
            if (board[i][j] == 1) {
                return false;
            }
            i--;
            j++;
        }

        return true;
    }

    // Solve N-Queens using backtracking
    static boolean solveNQueens(int[][] board, int row, int n) {

        // All queens are placed
        if (row == n) {
            return true;
        }

        // Try every column
        for (int col = 0; col < n; col++) {

            if (isSafe(board, row, col, n)) {

                // Place queen
                board[row][col] = 1;

                // Move to next row
                if (solveNQueens(board, row + 1, n)) {
                    return true;
                }

                // Backtrack
```

```
                board[row][col] = 0;
            }
        }

        return false;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the value of N: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Invalid input!");
            sc.close();
            return;
        }

        int[][] board = new int[n][n];

        System.out.println("Enter position of first queen:");

        System.out.print("Row (1 to N): ");
        int r = sc.nextInt() - 1;

        System.out.print("Column (1 to N): ");
        int c = sc.nextInt() - 1;

        // Validate position
        if (r < 0 || r >= n || c < 0 || c >= n) {
            System.out.println("Invalid position!");
        } else {

            // Place first queen
            board[r][c] = 1;

            // Check first queen and solve
            if (solveNQueens(board, r + 1, n)) {
                System.out.println("\nSolution:");
                printBoard(board, n);
            } else {
                System.out.println("No solution exists for this position.");
            }
        }

        sc.close();
    }
}
```

A5 - DAA

```
// QuickSort.java
import java.util.Arrays;
import java.util.Random;
import java.util.Scanner;
public class QuickSort {
    static Random random = new Random();


static int partitionDeterministic(int[] arr, int low, int high) {

    int pivot = arr[high];

    int i = low - 1;

    for (int j = low; j < high; j++) {

        if (arr[j] <= pivot) {

            i++;

            int temp = arr[i];

            arr[i] = arr[j];

            arr[j] = temp;

        }

    }

    int temp = arr[i + 1];

    arr[i + 1] = arr[high];

    arr[high] = temp;

    return i + 1;

}

static void deterministicQuickSort(int[] arr, int low, int high){
if (low < high) {
            int pivotIndex = partitionDeterministic(arr, low, high);

        deterministicQuickSort(arr, low, pivotIndex - 1);

        deterministicQuickSort(arr, pivotIndex + 1, high);

    }

}

static int partitionRandomized(int[] arr, int low, int high){
        int randomIndex = low + random.nextInt(high - low + 1);

    int temp = arr[randomIndex];

    arr[randomIndex] = arr[high];

    arr[high] = temp;

    return partitionDeterministic(arr, low, high);

}

static void randomizedQuickSort(int[] arr, int low, int high) {

    if (low < high) {

        int pivotIndex = partitionRandomized(arr, low, high);

        randomizedQuickSort(arr, low, pivotIndex - 1);

        randomizedQuickSort(arr, pivotIndex + 1, high);

    }

}
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of elements: ");
    int n = sc.nextInt();
    int[] arr = new int[n];
    System.out.println("Enter array elements:");
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }
    int[] arr1 = arr.clone();
    int[] arr2 = arr.clone();
    // Deterministic Quick Sort
    long start1 = System.nanoTime();
    deterministicQuickSort(arr1, 0, n - 1);
    long end1 = System.nanoTime();
    // Randomized Quick Sort
    long start2 = System.nanoTime();
    randomizedQuickSort(arr2, 0, n - 1);
    long end2 = System.nanoTime();
    System.out.println("\nDeterministic Quick Sort:");
    System.out.println(Arrays.toString(arr1));
    System.out.println("Execution Time: " + (end1 - start1) + " ns");
    System.out.println("\nRandomized Quick Sort:");
    System.out.println(Arrays.toString(arr2));
    System.out.println("Execution Time: " + (end2 - start2) + " ns");
    sc.close();
}
}
```

B1 - Machine Learning

```
# Uber fare prediction / dataset filename in code
# B1: UBER FARE PREDICTION

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# 1. Load dataset
df = pd.read_csv("uber.csv")

print("First five rows:")
print(df.head())

print("\nDataset information:")
print(df.info())

# 2. Remove unnecessary columns
df = df.drop(columns=["Unnamed: 0", "key"], errors="ignore")

# 3. Handle missing values
df = df.dropna()

# 4. Convert pickup_datetime
df["pickup_datetime"] = pd.to_datetime(
    df["pickup_datetime"], errors="coerce", utc=True
)

df = df.dropna(subset=["pickup_datetime"])

# Extract pickup hour
df["time"] = df["pickup_datetime"].dt.hour

df = df.drop(columns=["pickup_datetime"])

# 5. Remove invalid values
df = df[
    (df["fare_amount"] > 0) &
    (df["passenger_count"] > 0) &
    (df["passenger_count"] <= 6) &
    (df["pickup_longitude"].between(-180, 180)) &
    (df["dropoff_longitude"].between(-180, 180)) &
    (df["pickup_latitude"].between(-90, 90)) &
    (df["dropoff_latitude"].between(-90, 90))
]

# 6. Remove fare outliers using IQR
Q1 = df["fare_amount"].quantile(0.25)
Q3 = df["fare_amount"].quantile(0.75)
IQR = Q3 - Q1

df = df[
    (df["fare_amount"] >= Q1 - 1.5 * IQR) &
    (df["fare_amount"] <= Q3 + 1.5 * IQR)
]

# 7. Correlation heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

# 8. Separate features and target
X = df.drop("fare_amount", axis=1)
y = df["fare_amount"]

# 9. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 10. Feature scaling
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 11. Linear Regression
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)

lr_pred = lr.predict(X_test_scaled)

# 12. Random Forest Regression
rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

# 13. Evaluation
print("\nLINEAR REGRESSION")
print("R2 Score:", r2_score(y_test, lr_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, lr_pred)))
print("MAE:", mean_absolute_error(y_test, lr_pred))

print("\nRANDOM FOREST REGRESSION")
print("R2 Score:", r2_score(y_test, rf_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, rf_pred)))
print("MAE:", mean_absolute_error(y_test, rf_pred))

# 14. Actual vs Predicted graph
plt.figure(figsize=(8, 5))
plt.scatter(y_test, rf_pred, alpha=0.3)
plt.xlabel("Actual Fare")
plt.ylabel("Predicted Fare")
plt.title("Random Forest: Actual vs Predicted")
plt.show()
```

B2 - Machine Learning

```
# Email spam classification / dataset filename in code
# B2: EMAIL SPAM CLASSIFICATION USING KNN AND SVM

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# 1. Load dataset
df = pd.read_csv("emails.csv")

print(df.head())
print(df.shape)
print(df.info())

# 2. Remove unnecessary ID columns
df = df.drop(
    columns=["Email No.", "Email No", "Unnamed: 0"],
    errors="ignore"
)

# 3. Identify target column
possible_targets = [
    "Prediction", "Category", "label", "Label",
    "spam", "Spam", "target", "class", "Class"
]

target = next(
    (col for col in possible_targets if col in df.columns),
    df.columns[-1]
)

print("Target column:", target)

# 4. Separate features and target
X = df.drop(columns=[target])
y = df[target]

# Convert categorical labels if necessary
if y.dtype == "object":
    y = y.astype(str).str.lower().str.strip()
    y = y.map({
        "ham": 0,
        "not spam": 0,
        "spam": 1
    }).fillna(y)

# Convert features to numeric
X = X.apply(pd.to_numeric, errors="coerce")
X = X.fillna(X.median()).fillna(0)

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 6. Feature scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 7. KNN Classifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
knn_pred = knn.predict(X_test_scaled)
```

```
# 8. SVM Classifier
svm = SVC(kernel="linear")
svm.fit(X_train_scaled, y_train)

svm_pred = svm.predict(X_test_scaled)

# 9. Evaluation - KNN
print("\nKNN CLASSIFIER")
print("Accuracy:", accuracy_score(y_test, knn_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, knn_pred))
print(classification_report(y_test, knn_pred))

# 10. Evaluation - SVM
print("\nSVM CLASSIFIER")
print("Accuracy:", accuracy_score(y_test, svm_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, svm_pred))
print(classification_report(y_test, svm_pred))

# 11. Compare accuracy
models = ["KNN", "SVM"]
accuracies = [
    accuracy_score(y_test, knn_pred),
    accuracy_score(y_test, svm_pred)
]

plt.bar(models, accuracies)
plt.xlabel("Algorithms")
plt.ylabel("Accuracy")
plt.title("KNN vs SVM Accuracy")
plt.show()
```

B3 - Machine Learning

```
# Bank customer churn / Churn_Modelling.csv

# B3: BANK CUSTOMER CHURN PREDICTION USING ANN

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 1. Load dataset
df = pd.read_csv("Churn_Modelling.csv")

print(df.head())
print(df.info())

# 2. Drop unnecessary columns
df = df.drop(
    columns=["RowNumber", "CustomerId", "Surname"],
    errors="ignore"
)

# 3. Encode categorical columns
le = LabelEncoder()

df["Gender"] = le.fit_transform(df["Gender"])

df = pd.get_dummies(
    df,
    columns=["Geography"],
    drop_first=True,
    dtype=int
)

# 4. Separate features and target
X = df.drop("Exited", axis=1)
y = df["Exited"]

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 6. Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 7. Build Artificial Neural Network
model = Sequential()

model.add(Dense(16, activation="relu", input_shape=(X_train.shape[1],)))
model.add(Dense(8, activation="relu"))
model.add(Dense(1, activation="sigmoid"))

# 8. Compile model
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# 9. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
# 10. Predict
y_prob = model.predict(X_test)
y_pred = (y_prob > 0.5).astype(int).ravel()

# 11. Evaluate
print("\nANN ACCURACY:", accuracy_score(y_test, y_pred))

print("\nCONFUSION MATRIX:")
print(confusion_matrix(y_test, y_pred))

print("\nCLASSIFICATION REPORT:")
print(classification_report(y_test, y_pred))

# 12. Plot accuracy
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("ANN Accuracy")
plt.legend()
plt.show()
```

B4 - Machine Learning

```
# Gradient descent
```

```
# B4: GRADIENT DESCENT

import matplotlib.pyplot as plt

# 1. Define function
def function(x):
    return (x + 3) ** 2

# 2. Define derivative
def derivative(x):
    return 2 * (x + 3)

# 3. Initialize parameters
x = 2
learning_rate = 0.1
iterations = 20

# Store values
x_values = []
y_values = []

# 4. Gradient Descent
for i in range(iterations):

    gradient = derivative(x)

    x = x - learning_rate * gradient

    y = function(x)

    x_values.append(x)
    y_values.append(y)

    print(
        "Iteration:", i + 1,
        "x:", round(x, 4),
        "y:", round(y, 4)
    )

# 5. Final result
print("\nFinal x:", x)
print("Minimum y:", function(x))

# 6. Plot function
import numpy as np

x_plot = np.linspace(-6, 4, 100)
y_plot = function(x_plot)

plt.plot(x_plot, y_plot, label="y = (x+3)^2")
plt.scatter(x_values, y_values, color="red", label="Gradient Descent")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Gradient Descent Optimization")
plt.legend()
plt.grid()
plt.show()
```

B5 - Machine Learning

```
# Sales clustering / sales_data_sample.csv
# B5: SALES DATA CLUSTERING

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# 1. Load dataset
df = pd.read_csv(
    "sales_data_sample.csv",
    encoding="latin1"
)

print(df.head())
print(df.info())

# 2. Select numerical columns
features = [
    "QUANTITYORDERED",
    "PRICEEACH",
    "SALES",
    "MSRP"
]

X = df[features].copy()

# 3. Handle missing values
X = X.fillna(X.median())

# 4. Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 5. Elbow Method
wcss = []

for i in range(1, 11):

    kmeans = KMeans(
        n_clusters=i,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)

    wcss.append(kmeans.inertia_)

# 6. Plot Elbow graph
plt.plot(range(1, 11), wcss, marker="o")

plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()

# 7. Apply K-Means
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

df["KMeans_Cluster"] = kmeans.fit_predict(X_scaled)

print("\nK-Means Cluster Results:")
print(df[features + ["KMeans_Cluster"]].head())

# 8. Hierarchical Clustering
# Use a sample for dendrogram to keep execution fast
sample_size = min(500, len(X_scaled))

sample = X_scaled[:sample_size]
# 9. Create dendrogram
```

```
linked = linkage(sample, method="ward")

plt.figure(figsize=(12, 6))

dendrogram(linked)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Euclidean Distance")
plt.show()

# 10. Apply Agglomerative Clustering
hc = AgglomerativeClustering(
    n_clusters=3,
    linkage="ward"
)

df["Hierarchical_Cluster"] = hc.fit_predict(X_scaled)

print("\nHierarchical Clustering Results:")
print(df[features + ["Hierarchical_Cluster"]].head())

# 11. Visualize K-Means clusters
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="SALES",
    y="QUANTITYORDERED",
    hue="KMeans_Cluster",
    palette="viridis"
)

plt.title("K-Means Sales Clusters")
plt.show()
```

C1 - Blockchain

```
# BankAccount.sol
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract BankAccount {

    // Mapping to store balance of each customer address
    mapping(address => uint256) private balances;

    // Events
    event Deposited(address indexed customer, uint256 amount);
    event Withdrawn(address indexed customer, uint256 amount);

    // 1. Deposit Money
    function deposit() public payable {
        require(msg.value > 0, "Deposit amount must be greater than 0");
        balances[msg.sender] += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    // 2. Withdraw Money
    function withdraw(uint256 amount) public {
        require(amount > 0, "Withdraw amount must be greater than 0");
        require(balances[msg.sender] >= amount, "Insufficient balance");

        balances[msg.sender] -= amount;
        payable(msg.sender).transfer(amount);
        emit Withdrawn(msg.sender, amount);
    }

    // 3. Show Balance
    function showBalance() public view returns (uint256) {
        return balances[msg.sender];
    }
}
```

C2 - Blockchain

```
# StudentData.sol
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentData {

    struct Student {
        uint id;
        string name;
        uint age;
        string course;
        uint marks;
    }

    Student[] public students;

    event StudentAdded(
        uint id,
        string name,
        uint age,
        string course,
        uint marks
    );

    function addStudent(
        string memory _name,
        uint _age,
        string memory _course,
        uint _marks
    ) public {
        uint newId = students.length + 1;

        students.push(
            Student(newId, _name, _age, _course, _marks)
        );

        emit StudentAdded(
            newId,
            _name,
            _age,
            _course,
            _marks
        );
    }

    function getStudent(uint _index)
        public
        view
        returns (
            uint id,
            string memory name,
            uint age,
            string memory course,
            uint marks
        )
    {
        require(
            _index < students.length,
            "Index out of bounds"
        );

        Student memory s = students[_index];

        return (
            s.id,
            s.name,
            s.age,
            s.course,
            s.marks
        );
    }

    function getStudentCount() public view returns (uint) {
        return students.length;
    }

    function showBalance() public view returns (uint) {
        return address(this).balance;
    }
```

```
}
```